

ECE 473/573
Cloud Computing and Cloud Native Systems
Lecture 10 SQL Database II

Professor Jia Wang
School of Engineering, Applied Science, and Health
Illinois Institute of Technology

September 21, 2026

Outline

Transactions

Implementing Transactions

Reading Assignment

- ▶ This lecture: SQL Database
 - ▶ Go database queries: <https://go.dev/doc/database/querying>
 - ▶ Project 2 documentations and source code.
- ▶ Next lecture: 4

Outline

Transactions

Implementing Transactions

Buying a Listing

State	Before	After
Bob's balance	100	75
Alice's balance	100	125
Listing status	available	sold
Completed order	absent	present

- ▶ Bob buys Alice's used textbook for 25 credits.
 - ▶ Debit Bob's account and credit Alice's account.
 - ▶ Mark the listing sold and insert a completed order.
- ▶ One REST request changes several database rows.
 - ▶ What if another buyer tries to purchase the same listing?
 - ▶ What if Bob tries to purchase an item worth 100 credits at the same time?
 - ▶ What if the service fails between two updates?

Marketplace Invariants

- ▶ Invariants: conditions that must hold before and after one or a sequence of operations, e.g. for a successful purchase:
 - ▶ Balances remain nonnegative.
 - ▶ The buyer's debit equals the seller's credit.
 - ▶ A listing is sold at most once.
 - ▶ The sale and the completed order are recorded.
- ▶ Some conditions are enforced by database constraints.
 - ▶ CHECK on balances and UNIQUE on orders.listing_id.
- ▶ Others require correct transaction logic.
 - ▶ A database cannot infer that a missing seller credit is a bug.

ACID Guarantee

- ▶ Database updates are grouped into transactions to support application logic.
 - ▶ E.g. modify multiple tables to complete the purchase.
- ▶ **A**tomicity: either the transaction succeeds or fails as a whole.
 - ▶ Transactions are committed if they succeed.
 - ▶ Failed transactions should not change the database, e.g. if there is a power failure before crediting Alice's account after debiting Bob's account, then there should be a mechanism to rollback the updates to Bob's account.
- ▶ **C**onsistency: preserve the application's invariants.
 - ▶ Committed transactions should update the database correctly, preserving the application's invariants, e.g. Bob cannot have a negative balance.

ACID Guarantee (Cont.)

- ▶ (Serializable) **I**solaton: multiple transactions are executed as if sequentially (one after another).
 - ▶ Later transactions will see the changes from previously committed transactions.
 - ▶ Actual implementations may execute transactions concurrently to achieve better performance – but the outcome should be the same as if the transactions are executed one after another, while the order is not specified.
 - ▶ E.g. if Bob tries to purchase a 100-credit item at the same time, he should get the textbook or the item, but not both.
 - ▶ Other isolation levels exist though we focus on serializable.
- ▶ **D**urability: committed transactions survive system failures.
 - ▶ Usually by storing data on a drive.
 - ▶ To the extent that the drive won't fail.
- ▶ It is quite a challenge to achieve ACID at the same time.
 - ▶ E.g. what if there is a power failure when the database writes data to the disks to commit one transaction?

Outline

Transactions

Implementing Transactions

Implementing a Transaction in Go

```
tx, err := s.db.BeginTx(  
    context.Background(),  
    &sql.TxOptions{Isolation: sql.LevelSerializable},  
)  
if err != nil {  
    return Order{}, fmt.Errorf("begin checkout: %w", err)  
}  
defer tx.Rollback()  
... // use tx for database operations  
if err := tx.Commit(); err != nil {  
    return Order{}, fmt.Errorf("commit checkout: %w", err)  
}  
... // format and return the order
```

- ▶ `BeginTx()` requests serializable isolation and returns a transaction handle.
- ▶ `defer tx.Rollback()` provides cleanup on every return.
 - ▶ It does nothing if `tx.Commit()` succeeds.
 - ▶ Indeed, unless `tx.Commit()` succeeds, the database cleans up even if a power failure prevents to call `tx.Rollback()`.

Reading and Locking the Listing

```
SELECT listing_id, seller_id, title, credit_cost, status, listed_at  
FROM listings  
WHERE listing_id = $1  
FOR UPDATE
```

- ▶ **FOR UPDATE** locks the selected row.
 - ▶ By using a lock for each row, database implementations enable concurrent operations on multiple rows and tables.
- ▶ When may happen if we don't lock the listing row?

Two Buyers, One Listing

Step	Bob's request	Carol's request
1	Listing available	
2		Listing available
3	Attempt purchase	
4		Attempt purchase

- ▶ Without locking the row, both requests may find the listing available and then attempt to purchase the item.
- ▶ With the lock and by requesting serializable isolation, the one gets the lock could commit; the other will fail.

Reading and Locking the Accounts

```
SELECT balance
FROM accounts
WHERE account_id = $1
FOR UPDATE
```

- ▶ Lock the buyer account to prevent double spending.
 - ▶ What if Bob tries to buy a 100-credit item at the same time?
- ▶ Lock the seller account to prevent lost of credits.
 - ▶ What if another user tries to buy Alice's other item at the same time?
- ▶ Who's account should be locked first?

Deadlocks

- ▶ Suppose Alice and Bob buy each other's listing.
 - ▶ Assume the transaction will always lock buyer's account first.
 - ▶ A transaction locks Alice's account first and then Bob's.
 - ▶ A transaction locks Bob's account first and then Alice's.
 - ▶ If both transactions run concurrently and obtain their first locks, then neither of them can proceed because the other holds the second lock.
- ▶ Deadlock: a cycle of transactions waiting for each other.
- ▶ Deadlock prevention: transactions need to lock the rows in a specific order when updating a database, e.g.
 - ▶ First lock the row for listing.
 - ▶ Then lock the rows for the accounts following the dictionary order of the account names.
- ▶ Deadlock resolution: databases can detect deadlocks and abort a transaction. However,
 - ▶ The applications must handle the error properly.

Validating Marketplace Invariants

- ▶ The listing should be available (not sold yet).
- ▶ The buyer should be different than the seller.
- ▶ The buyer should have enough balance.
- ▶ Return immediate with an error message for violations.
 - ▶ Recall that `tx.Rollback()` will cleanup automatically.

Updating Marketplace State

```
UPDATE accounts  
SET balance = balance - $1  
WHERE account_id = $2
```

```
UPDATE accounts  
SET balance = balance + $1  
WHERE account_id = $2
```

```
UPDATE listings SET status = 'sold' WHERE listing_id = $1
```

- ▶ As part of the transaction.
- ▶ Check every error and return immediately on failure.
 - ▶ Allow `tx.Rollback()` to cleanup automatically.
 - ▶ E.g. even if the first debit succeeded, rollback will undo it if a later step fails.

Recording the Completed Order

```
INSERT INTO orders (  
    listing_id, buyer_id, seller_id, title, credit_cost,  
    status, listed_at, processed_at  
)  
VALUES ($1, $2, $3, $4, $5, 'completed', $6,  
        statement_timestamp())  
RETURNING order_id, listing_id, buyer_id, seller_id, ...
```

- ▶ Use **RETURNING** to read details from the inserted row.
- ▶ An error also rolls back the balance and listing updates.

Summary

- ▶ Transactions group database updates into one unit of work.
- ▶ Constraints and business logic preserve application invariants.
- ▶ Be careful with locks, plan ahead and handle failures properly.